

PilotDeck 现场演示与答辩手册

方向三 · 子任务双层验收、局部修复与验收经验记忆

版本：2026-09-11。主展示是改版 PilotDeck 本体。建议预留 5-7 分钟；模型响应速度不固定，90 秒版本用于讲解已有证据。

1. 先记住这三句话

子任务说“完成”，不等于交付合格。我们让 PilotDeck 检查交付声明、实际文件和原任务要求，不合格就在原子任务内修复。
第一层检查可执行契约；第二层由可独立配置的模型读取真实产物，核对业务要求。已经成功的兄弟任务保留结果。
验收结束后，框架把可核对的结果元数据写进项目白盒记忆，保留失败样本和分母。经验可以被整理和召回，但不会自动降低验收标准。

2. 新想法的客观判断

值得实现，但应当作为主线的增强，而不是改成另一个大项目。

真实缺口：验收报告已经保存在任务轨迹里，但原生记忆的消息归一化主要保留用户和助手文本，过滤工具消息。结构化 attempts、issues、repair 数据不能可靠地成为可检索的项目经验。直接记录验收结束事件可以补上这个缺口，不必让模型再读长日志、猜测发生过什么。

真正价值是：下次能够知道同类契约曾在哪些字段出问题、样本有多少、是否修复成功；开发者也能审查这些判断的来源。它比“助手自述自己学到了什么”更接近可核对事实。

现在不能声称形成了护城河，更不能声称已经提高成功率。基于反馈积累经验并非新概念，2023 年已有 [Reflexion](#) 和 [ExpeL](#)。这些研究给出方向上的依据，不证明本项目的具体实现有效。我们的贡献是把原生验收状态、局部修复轨迹与 PilotDeck 已有白盒记忆接通，并明确数据边界。

当前只做：记录、去重、分组、展示、供原生记忆整理与召回。暂不做自动改写契约、自动选择“更优秀模型”、根据一次失败生成全局规则。后续需要跨任务、跨模型的留出实验，证明召回经验后收益超过额外成本，才推进自动契约建议。

3. 现场前一天准备

安装与检查

使用公开仓库 `changer-changer/PilotDeck-Verified-Subtasks`，Node.js 22.13-22.x、pnpm 10.32.1。先完成安装、构建，避免现场下载依赖。

```
git clone https://github.com/changer-changer/PilotDeck-Verified-Subtasks.git
cd PilotDeck-Verified-Subtasks
pnpm install --frozen-lockfile
npm run build
pnpm --dir ui run build
```

公开复现：在本地终端配置 `PILOTDECK_DEMO_BASE_URL`、`PILOTDECK_DEMO_API_KEY`、`PILOTDECK_DEMO_MODEL`，模型必须支持工具调用。本机比赛凭据已单独保存在用户私密配置目录，可以显式使用 `--competition-auth`。密钥不在仓库、示例任务或公开证据中；投屏前关闭凭据编辑窗口。

跑一次完整预演

本机比赛环境：

```
node --import tsx scripts/verified-subtasks-native.ts live --competition-auth --semantic-fault --acceptance-memory
```

其他机器使用上述三个环境变量，省略 `--competition-auth`。每次启动都会创建新的 `artifacts/native-*/`，不会覆盖旧证据。成功预演应输出目录和核对结果；失败时保留该目录，不把失败轨迹换名成成功。

预演自动检查：四份交付最终通过；简报被模型读取后因语义问题拒绝并修复；三份正确报表哈希和修改时间不变；验收元数据进入本项目的原生记忆。具体结果见同目录的 `native-results.json`、`gateway-events.json`、`acceptance-memory.json`。本轮成功预演用时 215.664 秒，简报经历 1 次格式修复及 1 次语义修复，最终 4/4 通过、4 条观察入库；这是一例闭环，不是成功率估计。

用原生界面排练

```
node --import tsx scripts/verified-subtasks-native.ts ui --competition-auth --semantic-fault --acceptance-memory
```

等待前端与网关启动。在浏览器打开启动日志中的本机地址，默认前端为 `http://127.0.0.1:5173`。选择启动器打印的 `workspace` 完整路径，新建任务，复制该目录上一级的 `现场任务.txt`。按正常提示允许本演示所需的子任务和文件操作。

逐项人工确认：项目可选、交付评审设置可保存、任务可提交、子任务状态可展开、文件可查看、Memory 面板可找到“子任务验收经验”。若上一轮服务仍在运行，先在它的终端停止再启动，避免端口冲突。

当前验证边界：本轮自动浏览器访问被管理策略检查拦截，尚未完成浏览器点击和视觉验收；网关测试、组件测试和前端构建不能替代这一步。上台前务必在演示电脑按以上清单人工跑通。

4. 五至七分钟主展示：按这个顺序操作

0:00-0:40 · 让评委看见真实问题

打开 workspace/briefing-source.json 和 workspace/briefing.json。

源数据：远帆供应商延期 7 天，影响 18 单；由 采购负责人在 下周一 12:00前确认备选供应商交期。

当前简报：字段 summary、risk、action 都在，但内容是“供应稳定”“无风险”“无需跟进”。

说：“这份 JSON 完全可以通过结构检查，但业务结论错了。我们明确预置这个错误，所有评审和修复都由现场模型执行。这个实验说明机制能否闭环，不代表自然任务的错误分布。”

0:40-1:15 · 展示两层验收与可配置模型

打开 设置 → Agent → 交付评审。

展示评审开关、默认跟随主对话模型、可单独选择评审模型、轮次及超时设置。再展示“保存验收经验到项目记忆”开关；它需要先在 Agent → 记忆开启白盒记忆。本演示启动器已配置开启。

说：“默认复用本轮主对话实际采用的模型，也能独立配置。评审者读原任务、交付声明和真实产物，只能使用只读检查工具，不能自己把文件修好后说通过。”

评审读取范围仍受原有限权约束。界面配置不等于强制给所有旧任务加验收：本功能由 agent 调用中的 acceptance 契约启用。

1:15-3:45 · 提交并观察拒绝与局部修复

粘贴 现场任务.txt，提交。任务并行派发三份正确报表和一份错误简报。模型速度不同，此段时间可延长。

展开简报子任务，观察 验证 → 模型复核 → 修复 → 再次验收。模型如果额外提交了错误字段，可能先出现一次格式修复；此时说明第一层也在发挥作用，业务文件仍保持预置错误，之后继续观察语义复核。重点展示评审读过简报及源数据，以及指出的延期、订单影响和行动要求。不要只盯最终绿色状态。

说：“第一层确认结构和声明与文件一致；简报没有预设标准答案。第二层读取实际文件，按业务要求拒绝。框架把具体问题交回同一个子任务，不重新派发整个任务组。”

等待时可以打开验收配置说明：本例三个数值报表有宿主固定期望值；简报使用 matchClaimOnly 检查声明与文件一致，业务质量由第二层判断。它不是拿一个预写答案冒充模型理解。

3:45-4:40 · 看真实产物和没有重做的任务

打开修复后的 briefing.json：核对 7 天、18 单、采购负责人、下周一 12:00、备选供应商交期。然后查看三份报表仍然正确：销售总额 186、库存 48、退款 12。

说：“这些文件是真实交付。完成报告与验收结果分开记录；失败预算耗尽时会明确拒绝，不会无限循环。”

现场 UI 实跑可以查看文件与时间；自动哈希及修改时间断言来自 live 预演输出。不要把预演的断言包装成当前 UI 会话已经自动运行的检查。两者使用同一原生网关路径。

4:40-5:40 · 展示新增白盒记忆

打开当前项目的 Memory 面板，查找并打开 子任务验收经验。刷新后查看样本总数、首次通过、最终通过、未解决、基础设施错误、契约指纹和真实模型来源。

说：“这条经验不是让模型自由总结的，它来自框架验收结束事件。成功和失败都保留，每条样本可去重，字段问题带分母。评审调用不算新的生产任务。”

解释两份记录：SQLite 保留有界、结构化原始观察；白盒 Markdown 是供人和原生记忆系统读取的摘要。Dream 可以整理摘要；不会改变数据库中的观察。下一次记录会重新生成摘要。清空项目记忆也会清空原始观察。

不必现场运行 Dream：它是 PilotDeck 原生能力，会额外调用模型，耗时不稳定。若评委追问，可以展示 Dream 兼容测试与原生入口，明确本轮验证的是整理过程不破坏观察，而不是“已经学会并提高成功率”。

5:40-6:30 · 用证据收尾

打开公开仓库和验证文档，展示提交、测试与原始证据的位置。

说：“我们改的是执行链：契约、真实产物检查、独立复核、有界局部修复、状态与预算传播，再把验收观察接入已有记忆。现在证明的是闭环和边界；大样本成功率与记忆收益仍需后续实验。”

5. 九十秒精简版

1. 20 秒：展示结构正确、语义错误的原简报和源数据，说明受控故障注入。

2. 30 秒：展示一次真实预演的拒绝、同一子任务修复、最终通过轨迹；明确这是预演回放，不能称为九十秒现场实跑。

3. 20 秒：打开真实修复文件，指明业务要点和三份成功报表未变。

4. 20 秒：展示原生白盒验收经验及公开证据，说明新增桥接、可配置评审和有限预算。

6. 创新与工作量：让每句话都能落到证据

展示点	本项目新增工作	主要代码与证据
完成与验收分离	契约解析、支持的 schema 子集、拒绝原因、兼容旧协议	src/agent/sub/acceptance/; AcceptanceInvariants.spec.ts
真实产物检查	宿主注册、路径绑定、声明与文件比较、只读和体积边界	src/agent/sub/acceptance/artifactRegistry.ts; 原生文件验收测试
独立模型复核	任务与声明输入、真实文件独立读取、默认实际主模型或自选、只读工具集	src/agent/sub/acceptance/modelReviewer.ts; ModelReview*.spec.ts
局部修复	在原子任务继续, typed 提交边界, 共享轮次、取消、超时和用量	src/agent/sub/SubAgentSession.ts; VerifiedSubagent.spec.ts
原生体验	网关状态、前端验收状态、评审设置、双语文案	src/gateway/; UI bridge 与 reviewer settings 测试
验收经验桥接	终态观察、实际模型来源、脱敏、去重、窗口、原生 feedback 投影、清除 /Dream 兼容	src/context/memory/AcceptanceMemory.ts; AcceptanceObservation.spec.ts、AcceptanceMemory.spec.ts
可复现证据	原生模型预演、受控基准、成功与失败轨迹、文件哈希核对、公开 CI	scripts/verified-subtasks-native.ts; VALIDATION.zh-CN.md

白盒记忆、Dream、主界面、基础模型运行时来自上游 PilotDeck。我们的新增贡献是验收闭环与集成，不把整个上游仓库的文件数或行数算成自己的工作量。建议使用上表讲工程难点，而不是报一个没有意义的总代码量。

架构口述：“主 Agent 下发契约 → 子 Agent typed 提交 → 结构及产物验收 → 只读模型复核 → 失败在原会话有限修复 → 最终状态回主任务；旁路观察器将结果写入项目数据库，并投影成原生反馈记忆。”

7. 哪些结果可以说，哪些不能说

证据	可以支持	不支持
确定性三任务实验： 整组重跑 15 次请求、 局部修复 9 次	该受控场景减少 40% 请求； 实际文件工具执行	所有真实任务省 40%， 或自然模型成功率提升
已有 GLM-5.3 原生语义预演： 160.474 秒，4/4 通过， 1 次修复	真模型、真实网关和文件的单例闭环； 三份成功文件不变	平均延迟、稳定成功率、 生产环境成本上限
本轮 GLM-5.3 记忆预演： 215.664 秒、4/4 通过、 2 次局部修复、4 条观察	观察可保存、原生可见、 有界、去重；不会改变验收结果	经验召回已提升跨任务表现， Dream 输出一定正确
schema、预算、取消、 路径、模型复核测试	测到的条件分支行为符合断言	完全安全、任意 JSON Schema 或任意工作流都支持

观察窗口仅包含产生验收终态报告的子任务；预检抛错、取消或未产出报告的异常不进入窗口，不能用它替代全部请求的可靠性统计。

新增观察写入本身不调用模型。开启原生白盒记忆后的检索、整理、Dream 仍可能调用模型并增加成本。模型复核也有额外调用。轮次与超时限制并不是严格 token 或金额上限。

验收报告中的子任务用量包含评审用量时，不能再加一遍评审小计。原生记忆模型调用属于另一条链，不应悄悄漏算成“整条工作流总成本”。详细测量口径见验证文档。

8. 常见答辩问题

和给主模型加一句“请自检”有什么区别？自检提示依赖模型自行遵守。这里由执行框架在提交边界拦截，读取可核对文件，记录通过或拒绝，在预算内驱动修复；评审也不能绕过结构检查。

评审模型也会错，结果凭什么可信？我们不把模型判断当真理。确定性断言与只读证据提供底座；评审必须实际读取产物，错误、无证据和超时不能变成通过。业务仍可配置更强的宿主检查器。真实准确率需要标注测试集。

默认主模型既生产又评审，有什么独立性？是独立上下文和只读权限下的复核，不是统计独立性，也不能保证消除同源偏差；可配置独立模型。默认复用已能工作模型，降低配置门槛。

是不是所有任务都被强制验收？不是。当前在 agent 调用中显式传 acceptance 启用。必须由宿主约束的业务规则应由宿主定义契约，不能让生产模型自由选择宽松标准。

局部修复会不会一直循环？修复次数、共享模型轮次、超时和取消都有终止条件，终态明确拒绝。宿主自定义检查器需要自行终止不响应取消的外部操作。

记忆里记录了什么？契约指纹、实际观察到的模型标识、字段/错误代码、各次验收状态、修复次数、用量和去重标识。新观察器不保存任务正文、文件内容、交付值、自由文本评语或密钥。原生会话记忆本来的记录策略不因此改变。

能判断“什么反馈有效”吗？当前只能说某字段问题反馈后，该次交付后来通过；没有对照实验，不能证明因果，也不保存自由文本反馈。这是诚实的阶段边界。

Dream 整理后原始数据会丢吗？派生 Markdown 可能被合并、改写或删除；数据库里的有界观察独立保存。清空项目记忆会一起删除。原生记忆导出包以文件为主，原始观察由演示导出的 JSON 单独提供。

为什么要保留失败和分母？只写成功会制造幸存者偏差；同一任务重复回放也不能算新样本。当前最多保留最近 128 条，统计描述的是这个窗口，不是全部历史。

跨进程是否绝对不会丢记录？同一原生进程内同步写入，按项目数据库隔离；未声称支持多个独立网关并发合并同一观察窗口。部署多写进程前需要增加事务级并发合并。

这是护城河吗？当前是有用的集成。长期积累可信、可复现实验和真实使用数据后才可能形成优势；功能本身可复制，不能靠一个记忆开关宣称壁垒。

9. 故障与备用流程

模型超时或网络中断：展示实际错误及预算停止，说明这也是可信执行的一部分。切到保存的成功预演目录，并明确说“下面是刚才预演的真实记录”。不要反复点提交产生多个重叠任务。

文件已经修好，想再演：停止当前演示服务，重新运行 `ui --semantic-fault --acceptance-memory` 创建全新目录，选择新项目。不要把上一次通过后的文件当作未修复输入。

记忆面板为空：先确认选中同一个 workspace；验收是否已结束；白盒记忆和保存开关是否开启；再刷新。观察写入失败会发 `acceptance_memory_capture_failed` 警告，不应把已通过任务改成失败。保留日志排查，不现场改验收标准。

完全离线：运行下面的机制实验和轨迹生成器，使用生成的 `index.html`。必须说明模型响应是脚本，文件工具是真实执行。

```
node --import tsx scripts/verified-subtasks-benchmark.ts artifacts/offline-demo
node --import tsx scripts/verified-subtasks-viewer.ts artifacts/offline-demo
```

备用回放证明机制与证据结构，不能冒充现场真实模型调用。界面遇到权限或管理策略阻断时，请操作者处理环境，不能用其他自动化通道绕过。

10. 提交与上台检查

方向三必备：公开 GitHub 代码仓链接、游园会海报；原生 PilotDeck 可用于选择性 Demo 及现场演示。

README 说明具体改进、架构和运行方式。性能对照、已知限制、技术图和视频属于加分材料；本项目不要用不可核对宣传替代证据。

- 已下载依赖、构建完成，模型凭据可用，原生界面人工完整排练。
- 准备全新演示目录，确认任务文件、故障文件、源数据、验收规则匹配。
- 准备真实成功预演及原始日志、可离线打开的回放、海报与本手册。
- 讲述中区分上游能力与新增工作、实跑与回放、单例闭环与统计收益。
- 最后让评委看到：具体问题、PilotDeck 改动位置、真实闭环、可信证据及限制。

公开代码：<https://github.com/changer-changer/PilotDeck-Verified-Subtasks>。完整验证入口为同目录 `VALIDATION.zh-CN.md`；简版运行说明为 `DEMO.zh-CN.md`。